Supervised Learning

Sanjay Singh

School of Computer Engineering
Manipal Institute of Technology, MAHE
Manipal-576104, INDIA
sanjay.singh@manipal.edu

January 27, 2026

Learning Problems

Major classes of learning problems are:

- Classification: Assign a category to each item
- Regression: Predict a real value for each item
- Ranking: Order items according to some criterion
- Clustering: Partition items into homogeneous regions usually performed on very large data sets
- Dimensionality reduction or manifold learning: Transform initial representation of items into a lower-dimensional representation of these items while preserving some properties of the initial representation

Main objective of ML is accurate prediction of unseen item and design efficient and robust algorithms to produce these predictions.

Definitions and Terminology

We'll consider an example of SPAM filter to comprehend these terms:

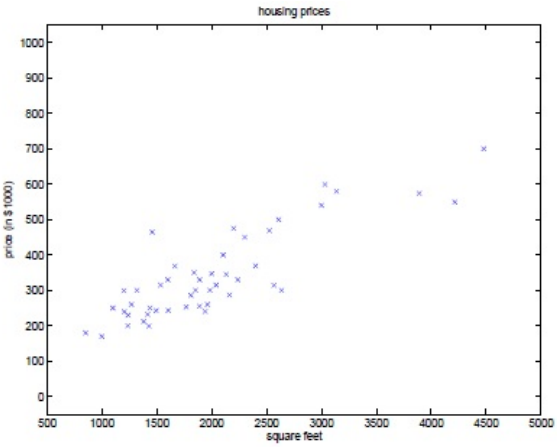
- *Example*: Items or instances of data used for learning or evaluation.
- *Features*: Set of attributes, often represented as a vector, associated to an example. E.g length of message, name of sender, presence of certain keywords and so on.
- *Labels*: Values or categories assigned to examples, e.g, SPAM and non-SPAM.
- *Training sample*: Examples used to train a learning algorithm.
- *Validation sample*: Examples used to tune the parameters of a learning algorithm when working with labeled data. Learning algorithms typically have one or more free parameters, and validation sample is used to select appropriate values for these model parameters.

Definitions and Terminology

- *Test sample*: Examples used to evaluate the performance of a learning algorithm. Test sample is separate from the training and validation data.
- *Loss function*: A function that measures the difference, or loss, between a predicted label and a true label.
  With $\mathcal{Y}$ as actual label and $\mathcal{Y}'$ as predicted label, a loss function is a mapping, $L : \mathcal{Y} \times \mathcal{Y}' \mapsto \mathbb{R}_+$.
- *Hypothesis set*: A set of functions mapping features (feature vectors) to the set of labels $\mathcal{Y}$.
  In our example, these may be a set of functions mapping email features to $\mathcal{Y} = \{\text{SPAM, non-SPAM}\}$.

Suppose we have a dataset giving the living areas and prices of 50 houses from Manipal

Living area ($feet^2$)	Price (10
2104	400
1600	330
2400	369
1416	232
3000	540
⋮	⋮



Given such data, how can we learn to predict prices of other houses in Manipal, as a function of size of their living area?

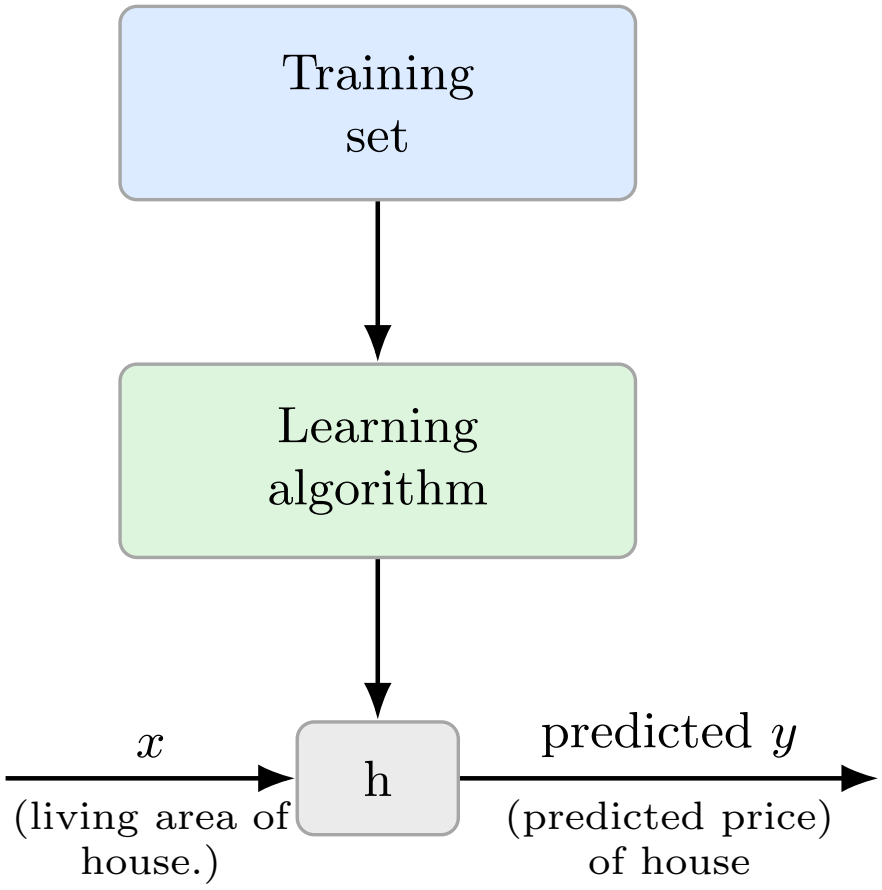


Figure: Pictorial representation of ML terminologies

Linear Regression

Consider the housing example, with $x \in \mathbb{R}^2$

- To perfofm supervised learning, decide about the hypothesis h
- As initial choice, decide y as linear combination of x

$$h_\theta(x) = \theta_0 + \theta_1 x_1 + \theta_2 x_2$$

- Here, θ_i are parameters (also called weights) parameterizing the space of linear function mapping from $\mathcal{X}$ to $\mathcal{Y}$
- To simplify our notation, we introduce $x_0 = 1$ (intercept term), so that

$$h(x) = \sum_{i=0}^{n} \theta_i x_i = \theta^T x$$

Given a training set, how to pick (learn), the parameter θ?

- Make $h(x)$ close to y for the training example
- Formally, define a function that measures this closeness by **cost function**

$$J(\theta) = \frac{1}{2} \sum_{i=1}^{m} (h(x^{(i)}) - y^{(i)})^2$$

- Least-square regression model

Least Mean Square (LMS)

We want to choose θ so as to minimize $J(\theta)$

- Start with some "initial guess" for θ and repeatedly changes θ to make smaller $J(\theta)$ until convergence
- Specifically, **gradient descent** algorithm starts with some initial θ and repeatedly performs the update:

$$\theta_j := \theta_j - \alpha \frac{\partial J(\theta)}{\partial \theta_j} \quad j = 0, \ldots, n$$

- GDA takes a step in the direction of steepest decrease of J

Consider a case with only one training example (x, y), so that we can neglect the sum in definition of J

- $$\begin{aligned} \frac{\partial J(\theta)}{\partial \theta_j} &= \frac{\partial}{\partial \theta_j} \frac{1}{2}(h(x) - y)^2 \\ &= 2.\frac{1}{2}(h(x) - y).\frac{\partial}{\partial \theta_j}(h(x) - y) \\ &= (h(x) - y).\frac{\partial}{\partial \theta_j}\left(\sum_{i=0}^{n} \theta_i x_i - y\right) \\ &= (h(x) - y)x_j \end{aligned}$$
- For a single training example, this gives the update rule

$$\theta_j := \theta_j + \alpha(y^{(i)} - h(x^{(i)}))x_j^{(i)}$$

- LMS update rule (also known as Widrow-Hoff learning rule)

LMS update rule for a training set of more than one example

- Repeat until convergence {
  $\theta_j := \theta_j + \alpha \sum_{i=1}^{m} (y^{(i)} - h(x^{(i)}))x_j^{(i)} \quad \forall j$
  }
- This method looks at every example in the set on every step, and called batch gradient descent

Stochastic Gradient Descent (SGD)

- Loop{
  for i=1 to m, {
  $\theta_j := \theta_j + \alpha(y^{(i)} - h(x^{(i)}))x_j^{(i)}$
  }
  }
- Also called incremental gradient descent
- BGD is costly if m is large
- When m is large, SGD is preferred over BGD

Example Problem Setup

Consider a simple linear regression problem with:

- One feature ($n = 1$), so hypothesis $h_\theta(x) = \theta_0 + \theta_1 x$
- Training data with $m = 3$ examples:

Table: Training examples

i	$x^{(i)}$	$y^{(i)}$
1	1	2
2	2	3
3	3	4

- Initial parameters: $\theta_0 = 0$, $\theta_1 = 0$
- Learning rate: $\alpha = 0.1$

Batch Gradient Descent Update Rule

For our linear regression problem:

$$J(\theta) = \frac{1}{2m} \sum_{i=1}^{m} (h_\theta(x^{(i)}) - y^{(i)})^2$$

$$h_\theta(x) = \theta_0 + \theta_1 x$$

Batch gradient descent updates:

$$\theta_0 := \theta_0 + \alpha \sum_{i=1}^{m} (y^{(i)} - h_\theta(x^{(i)}))$$

$$\theta_1 := \theta_1 + \alpha \sum_{i=1}^{m} (y^{(i)} - h_\theta(x^{(i)}))x^{(i)}$$

Iteration 0: Initial State

- Initial parameters: $\theta_0^{(0)} = 0$, $\theta_1^{(0)} = 0$
- Initial predictions:

i	$x^{(i)}$	$y^{(i)}$	$h_\theta(x^{(i)})$	Error $(y^{(i)} - h_\theta)$
1	1	2	0	2
2	2	3	0	3
3	3	4	0	4

$$\sum_{i=1}^{3}(y^{(i)} - h_\theta(x^{(i)})) = 2 + 3 + 4 = 9$$

$$\sum_{i=1}^{3}(y^{(i)} - h_\theta(x^{(i)}))x^{(i)} = 2 \times 1 + 3 \times 2 + 4 \times 3 = 20$$

Iteration 1: First Update

Using update rules:

$$\theta_0^{(1)} = \theta_0^{(0)} + \alpha \times 9 = 0 + 0.1 \times 9 = 0.9$$
$$\theta_1^{(1)} = \theta_1^{(0)} + \alpha \times 20 = 0 + 0.1 \times 20 = 2.0$$

New predictions:

i	$x^{(i)}$	$y^{(i)}$	$h_\theta(x^{(i)})$	Error $(y^{(i)} - h_\theta)$
1	1	2	$0.9 + 2 \times 1 = 2.9$	-0.9
2	2	3	$0.9 + 2 \times 2 = 4.9$	-1.9
3	3	4	$0.9 + 2 \times 3 = 6.9$	-2.9

$$\sum_{i=1}^{3} \text{Error} = -0.9 - 1.9 - 2.9 = -5.7$$

$$\sum_{i=1}^{3} \text{Error} \times x^{(i)} = -0.9 \times 1 - 1.9 \times 2 - 2.9 \times 3 = -14.6$$

Iteration 2: Second Update

$$\theta_0^{(2)} = 0.9 + 0.1 \times (-5.7) = 0.9 - 0.57 = 0.33$$
$$\theta_1^{(2)} = 2.0 + 0.1 \times (-14.6) = 2.0 - 1.46 = 0.54$$

New predictions:

i	$x^{(i)}$	$y^{(i)}$	$h_\theta(x^{(i)})$	Error $(y^{(i)} - h_\theta)$
1	1	2	$0.33 + 0.54 \times 1 = 0.87$	1.13
2	2	3	$0.33 + 0.54 \times 2 = 1.41$	1.59
3	3	4	$0.33 + 0.54 \times 3 = 1.95$	2.05

$$\sum_{i=1}^{3} \text{Error} = 1.13 + 1.59 + 2.05 = 4.77$$

$$\sum_{i=1}^{3} \text{Error} \times x^{(i)} = 1.13 \times 1 + 1.59 \times 2 + 2.05 \times 3 = 11.06$$

Iteration 3: Third Update

$$\theta_0^{(3)} = 0.33 + 0.1 \times 4.77 = 0.33 + 0.477 = 0.807$$
$$\theta_1^{(3)} = 0.54 + 0.1 \times 11.06 = 0.54 + 1.106 = 1.646$$

- After 3 iterations: $\theta_0 = 0.807$, $\theta_1 = 1.646$
- True optimal (analytical solution): $\theta_0 = 1$, $\theta_1 = 1$
- Cost function $J(\theta)$ is decreasing with each iteration

Iteration	$J(\theta)$	Change
0	$\frac{1}{6}(4 + 9 + 16) = 4.833$	-
1	$\frac{1}{6}(0.81 + 3.61 + 8.41) = 2.138$	$\downarrow$ 2.695
2	$\frac{1}{6}(1.2769 + 2.5281 + 4.2025) = 1.3346$	$\downarrow$ 0.8034
3	$\frac{1}{6}(1.404 + 0.858 + 0.362) = 0.437$	$\downarrow$ 0.8976

Key Observations

- LMS/Batch Gradient Descent successfully reduces $J(\theta)$ each iteration
- Parameters gradually approach optimal values
- Learning rate α controls step size:
  - Too large: may overshoot or diverge
  - Too small: slow convergence
- Each iteration uses ALL training examples (batch update)
- Algorithm continues until convergence (small parameter changes)

Update rule in action:

$$\theta_j := \theta_j + \alpha \sum_{i=1}^{m} (y^{(i)} - h_\theta(x^{(i)}))x_j^{(i)}$$

Some relation from matrix calculus

- $\nabla_x b^T x = b$
- $\nabla_x x^T A x = 2Ax$ (if A symmetric)
- $\nabla_x^2 x^T A x = 2A$ (if A symmetric)
- $\nabla_A trAB = B^T$
- $\nabla_{A^T} f(A) = (\nabla_A f(A))^T$
- $\nabla_A trABA^T C = CAB + C^T AB^T$
- $\nabla |A| = |A|(A^{-1})^T$

Least squares revisited-normal equation approach

Armed with tools of matrix derivative, lets find the closed-form value of θ that minimizes $J(\theta)$

- Write J in matrix-vectorial notation
- Given a training set, define design matrix X as

$$X = \begin{bmatrix} -(x^{(1)})^T- \\ -(x^{(2)})^T- \\ \vdots \\ -(x^{(m)})^T- \end{bmatrix}$$

- Let $\vec{y}$ is vector of all the target values

$$\begin{bmatrix} y^{(1)} \\ y^{(2)} \\ \vdots \\ y^{(m)} \end{bmatrix}$$

- Since $h(x^{(i)}) = (x^{(i)})^T\theta$, we can write

$$X\theta - \vec{y} = \begin{bmatrix} (x^{(1)})^T\theta \\ \vdots \\ (x^{(m)})^T\theta \end{bmatrix} - \begin{bmatrix} y^{(1)} \\ \vdots \\ y^{(m)} \end{bmatrix} = \begin{bmatrix} h(x^{(1)}) - y^{(1)} \\ \vdots \\ h(x^{(m)}) - y^{(m)} \end{bmatrix}$$

- Using the fact that for a vector z, $z^Tz = \sum_i z_i^2$:

$$\frac{1}{2}(X\theta - \vec{y})^T(X\theta - \vec{y}) = \frac{1}{2}\sum_{i=1}^{m}(h(x^{(i)}) - y^{(i)})^2 = J(\theta)$$

$$
\begin{aligned}
\nabla_\theta J(\theta) &= \nabla_\theta \frac{1}{2}(X\theta - \vec{y})^T(X\theta - \vec{y}) \\
&= \frac{1}{2}\nabla_\theta(\theta^T X^T - \vec{y}^T)(X\theta - \vec{y}) \\
&= \frac{1}{2}\nabla_\theta(\theta^T X^T X\theta - \theta^T X^T \vec{y} - \vec{y}^T X\theta + \vec{y}^T\vec{y}) \\
&= \frac{1}{2}\nabla_\theta\left(\theta^T X^T X\theta - (X^T\vec{y})^T\theta - \vec{y}^T X\theta + \vec{y}^T\vec{y}\right) \quad \text{using } A^TB = B^TA \\
&= \frac{1}{2}\nabla_\theta\left(\theta^T X^T X\theta - (X^T\vec{y})^T\theta - (X^T\vec{y})^T\theta + \vec{y}^T\vec{y}\right) \quad \text{using } (A^TB)^T = B^T(A \\
&= \frac{1}{2}\nabla_\theta\left(\theta^T X^T X\theta - 2(X^T\vec{y})^T\theta + \vec{y}^T\vec{y}\right) \\
&= \frac{1}{2}\left(\nabla_\theta(\theta^T X^T X\theta) - 2\nabla_\theta((X^T\vec{y})^T\theta) + \nabla_\theta(\vec{y}^T\vec{y})\right) \\
&= \frac{1}{2}\left(2X^T X\theta - 2\nabla_\theta((X^T\vec{y})^T\theta) + \nabla_\theta(\vec{y}^T\vec{y})\right) \quad \text{using } \nabla_x x^T Ax = 2Ax \\
&= \frac{1}{2}(2X^T X\theta - 2X^T\vec{y} + 0) \quad \text{using } \nabla_x b^T x = b \\
&= X^T X\theta - X^T\vec{y}
\end{aligned}
$$

To minimize J, its derivative is set to zero, and obtain the normal equations

$$X^T X\theta = X^T\vec{y}$$

Value of θ that minimizes J is given in the closed form as

$$\theta = (X^T X)^{-1}X^T\vec{y}$$

Also, we can write it as

$$\theta = X^+\vec{y}$$

Maximum Likelihood Estimate (MLE)

Principle of MLE

Maximum Likelihood Estimate (MLE) is a method of estimating the parameters of a (statistical) model given observation. MLE attempts to find the parameter values that maximize the likelihood function, given the observation. The resulting estimate is called a maximum likelihood estimate.

SC:Maximum Likelihood Estimation

Maximum likelihood principle

- Consider $\mathbb{X} = \{x^{(i)} | i = 1, \ldots m\}$ drawn independently from the true but unknown data generating distribution $p_{data}(x)$
- Let $p_{model}(x; \theta)$ be a parametric family of probability distributions over the same space indexed by θ
- $p_{model}(x; \theta)$ maps any configuration x to a real number estimating the true probability $p_{data}(x)$
- The maximum likelihood estimator for θ is defined as

$$\theta_{ML} = \underset{\theta}{argmax}\ p_{model}(X; \theta)$$

$$= \underset{\theta}{argmax} \prod_{i=1}^{m} p_{model}(x^{(i)}; \theta)$$

Probabilistic Interpretation

- Why least square cost function J be a reasonable choice?
- We'll give set of probabilistic assumptions, under which least-squares regression is derived
- Lets assume that, the target variable and inputs are related as

$$y^{(i)} = \theta^T x^{(i)} + \epsilon^{(i)}$$

  where $\epsilon^{(i)}$ is an error term that captures either unmodeled effects or random noise
- We assume that $\epsilon^{(i)}$ are distributed IID according to Gaussian distribution with zero mean and variance σ^2

- $\epsilon^{(i)} \sim \mathcal{N}(0, \sigma^2)$, the density of $\epsilon^{(i)}$ is given by

$$p(\epsilon^{(i)}) = \frac{1}{\sqrt{2\pi}\sigma} exp\left(-\frac{(\epsilon^{(i)})^2}{2\sigma^2}\right)$$

- This implies that

$$p(y^{(i)}|x^{(i)};\theta) = \frac{1}{\sqrt{2\pi}\sigma} exp\left(-\frac{(y^{(i)} - \theta^T x^{(i)})^2}{2\sigma^2}\right)$$

- The notation $p(y^{(i)}|x^{(i)};\theta)$ indicates that this is the distribution of $y^{(i)}$ given $x^{(i)}$ and parameterized by θ

- Given X, the design matrix and θ, what is the distribution of $y^{(i)}$'s?
- The probability of the data is given by $p(\vec{y}|X;\theta)$- which is viewed as a function of $\vec{y}$ for a fixed value of θ
- When we view it as a function of θ, we call it the **likelihood** function:

$$L(\theta) = L(\theta; X, \vec{y}) = p(\vec{y}|X;\theta)$$

- By the independent assumption of $\epsilon^{(i)}$ (and $y^{(i)}$'s given $x^{(i)}$'s), this can be written as

$$\begin{aligned} L(\theta) &= \prod_{i=1}^{m} p(y^{(i)}|x^{(i)};\theta) \\ &= \prod_{i=1}^{m} \frac{1}{\sqrt{2\pi}\sigma} exp\left(-\frac{(y^{(i)} - \theta^T x^{(i)})^2}{2\sigma^2}\right) \end{aligned}$$

- Given this probabilistic model relating $y^{(i)}$'s and $x^{(i)}$'s, what is the reasonable way of choosing best guess of the parameter θ?
- Principle of **maximum likelihood** says that we should choose θ so as to make data as high probable as possible i.e choose θ to maximize $L(\theta)$
- For ease of computation, we maximize the **log likelihood** $l(\theta)$

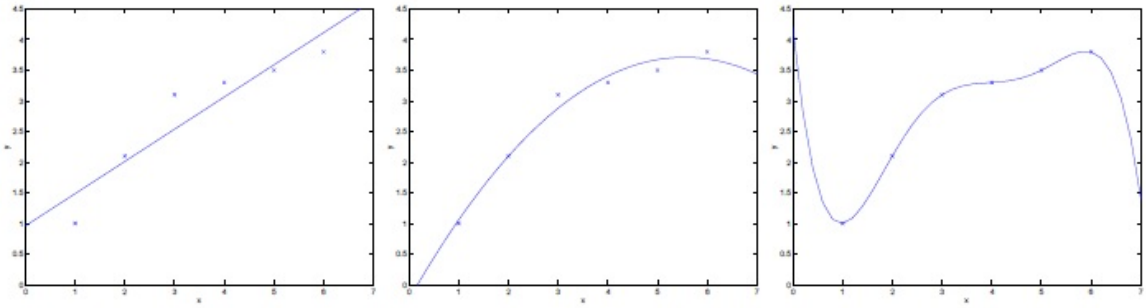
$$\begin{aligned} l(\theta) &= logL(\theta) \\ &= log \prod_{i=1}^{m} \frac{1}{\sqrt{2\pi}\sigma} exp\left(-\frac{(y^{(i)} - \theta^T x^{(i)})^2}{2\sigma^2}\right) \\ &= \sum_{i=1}^{m} log \frac{1}{\sqrt{2\pi}\sigma} exp\left(-\frac{(y^{(i)} - \theta^T x^{(i)})^2}{2\sigma^2}\right) \\ &= \sum_{i=1}^{m} \left(log \frac{1}{\sqrt{2\pi}\sigma} + \log \exp\left(-\frac{(y^{(i)} - \theta^T x^{(i)})^2}{2\sigma^2}\right)\right) \\ &= \sum_{i=1}^{m} log \frac{1}{\sqrt{2\pi}\sigma} + \sum_{i=1}^{m} \log \exp\left(-\frac{(y^{(i)} - \theta^T x^{(i)})^2}{2\sigma^2}\right) \\ &= m \log \frac{1}{\sqrt{2\pi}\sigma} - \frac{1}{\sigma^2}\frac{1}{2}\sum_{i=1}^{m}(y^{(i)} - \theta^T x^{(i)})^2 \end{aligned}$$

- Maximizing $l(\theta)$ gives the same answer as minimizing

$$\frac{1}{2}\sum_{i=1}^{m}(y^{(i)} - \theta^T x^{(i)})^2$$

  which is nothing but $J(\theta)$, the least-square cost function
- Summary-Under probabilitic assumption on data, least-square regression corresponds to finding the maximum likelihood estimate of θ

Locally weighted linear regression



- Consider the problem of predicting y from $x \in \mathbb{R}$
- $y = \theta_0 + \theta_1 x$ underfit-data clearly shows structure not captured by the model
- $y = \theta_0 + \theta_1 x + \theta_2 x^2$ fit
- $y = \sum_{j=0}^{5} \theta_j x^j$ Overfitting refers to a model that models the training data too well

- Choice of features is important to ensure good performance of a learning algorithm
- Locally weighted linear regression (LWR) algorithm which, assuming there is sufficient training data, makes the choice of features less critical
- In original linear regression, to make a prediction at a query point, we would:
  1. Fit θ to minimize $\sum_i (y^{(i)} - \theta^T x^{(i)})^2$
  2. Output $\theta^T x$
- In contrast, LWR algorithm does the following
  1. Fit θ to minimize $\sum_i w^{(i)} (y^{(i)} - \theta^T x^{(i)})^2$
  2. Output $\theta^T x$
- Where $w^{(i)}$'s are non-negative valued weights

- If $w^{(i)}$ is large for a particular value of i, then in picking θ, we try to make $(y^{(i)} - \theta^T x^{(i)})^2$ small
- If $w^{(i)}$ is small, then the error term $(y^{(i)} - \theta^T x^{(i)})^2$ will be ignored in the fit
- A standard choice for the weights is given by

$$w^{(i)} = exp\left(-\frac{(x^{(i)} - x)^2}{2\tau^2}\right)$$

- Weights depend on the particular query point x
- $w^{(i)} = \begin{cases} 1 & |x^{(i)} - x| \ small \\ small & |x^{(i)} - x| \ large \end{cases}$
- θ is chosen giving much higher weight to the training example close to the query point
- Parameter τ, called **bandwidth** controls how quickly the weight of a training example falls off with distance from the query point

Example on LWR

Problem Setup:

- We have 5 training examples:

$x^{(i)}$	$y^{(i)}$
1.0	2.0
2.0	3.5
3.0	5.0
4.0	6.5
5.0	8.0

- Query point: $x_q = 3.2$
- Bandwidth: $\tau = 1.5$
- Model: $y = \theta_0 + \theta_1 x$

Step 1: Calculate Weights

Weight function: $w^{(i)} = \exp\left(-\frac{(x^{(i)}-x_q)^2}{2\tau^2}\right)$

$x^{(i)}$	$(x^{(i)} - x_q)^2$	$w^{(i)}$
1.0	$(1.0-3.2)^2 = 4.84$	$\exp(-\frac{4.84}{2\times 2.25}) = 0.340$
2.0	$(2.0-3.2)^2 = 1.44$	$\exp(-\frac{1.44}{4.5}) = 0.726$
3.0	$(3.0-3.2)^2 = 0.04$	$\exp(-\frac{0.04}{4.5}) = 0.991$
4.0	$(4.0-3.2)^2 = 0.64$	$\exp(-\frac{0.64}{4.5}) = 0.867$
5.0	$(5.0-3.2)^2 = 3.24$	$\exp(-\frac{3.24}{4.5}) = 0.481$

Observation: Points closer to $x_q = 3.2$ have higher weights!

Step 2: Set Up Weighted Least Squares

We minimize: $J(\theta) = \sum_{i=1}^{5} w^{(i)}(y^{(i)} - \theta_0 - \theta_1 x^{(i)})^2$

i	$x^{(i)}$	$y^{(i)}$	$w^{(i)}$	$w^{(i)}y^{(i)}$
1	1.0	2.0	0.340	0.680
2	2.0	3.5	0.726	2.541
3	3.0	5.0	0.991	4.955
4	4.0	6.5	0.867	5.636
5	5.0	8.0	0.481	3.848

Step 3: Solve for θ

Normal equations for weighted regression:

$$\theta_0 \sum w^{(i)} + \theta_1 \sum w^{(i)} x^{(i)} = \sum w^{(i)} y^{(i)}$$
$$\theta_0 \sum w^{(i)} x^{(i)} + \theta_1 \sum w^{(i)} x^{(i)2} = \sum w^{(i)} x^{(i)} y^{(i)}$$

Let's compute the summations:

$$\sum w^{(i)} = 0.340 + 0.726 + 0.991 + 0.867 + 0.481 = 3.405$$
$$\sum w^{(i)} x^{(i)} = 0.340 \times 1 + 0.726 \times 2 + 0.991 \times 3 + 0.867 \times 4 + 0.481 \times 5$$
$$\sum w^{(i)} x^{(i)2} = 0.340 \times 1 + 0.726 \times 4 + 0.991 \times 9 + 0.867 \times 16 + 0.481 \times 2$$
$$\sum w^{(i)} y^{(i)} = 0.680 + 2.541 + 4.955 + 5.636 + 3.848 = 17.660$$
$$\sum w^{(i)} x^{(i)} y^{(i)} = 0.340 \times 2 + 0.726 \times 7 + 0.991 \times 15 + 0.867 \times 26 + 0.481 \times$$

Step 4: Solve the System

Our system of equations:

$$3.405\theta_0 + 10.388\theta_1 = 17.660$$
$$10.388\theta_0 + 35.854\theta_1 = 58.583$$

Solving this system:

$$\theta_1 = \frac{3.405 \times 58.583 - 10.388 \times 17.660}{3.405 \times 35.854 - 10.388 \times 10.388}$$
$$= \frac{199.478 - 183.482}{122.114 - 107.942} = \frac{15.996}{14.172} = 1.129$$
$$\theta_0 = \frac{17.660 - 10.388 \times 1.129}{3.405} = \frac{17.660 - 11.733}{3.405} = 1.741$$

Our locally weighted model at $x_q = 3.2$ is:

$$y = 1.741 + 1.129x$$

Step 5: Make Prediction

For query point $x_q = 3.2$:

$$y_q = \theta_0 + \theta_1 x_q = 1.741 + 1.129 \times 3.2 = 1.741 + 3.613 = 5.354$$

Comparison with ordinary linear regression:

- Ordinary LR gives: $y = 0.5 + 1.5x$ (equal weights for all points)
- Ordinary LR prediction: $0.5 + 1.5 \times 3.2 = 5.300$
- LWR prediction: 5.354
- Difference occurs because LWR gives more importance to points near $x_q = 3.2$

Insights from the Example

- **Weights vary with query point:** If x_q changes, we get different weights
- **Local adaptation:** The fitted line changes depending on which region we're querying
- **Bandwidth τ matters:**
  - Larger $\tau \rightarrow$ weights more uniform $\rightarrow$ behaves like ordinary regression
  - Smaller $\tau \rightarrow$ only very close points matter $\rightarrow$ more flexible but noisy
- **Computational cost:** Must solve a new regression for each query point!
- **Advantage:** Can fit complex patterns without high-order polynomials

Why use LWR?

- Avoids overfitting of high-degree polynomials
- Adapts to local patterns in data
- Only needs to choose bandwidth τ, not polynomial degree